



École Nationale des Sciences Appliquées

Architecture des Microservices

Docker Compose

Orchestration de microservices

FI	GI 2
Module	Microservice
AU	2025/2026

Réalisé par :
Wiam BenKrimen

Encadré par :
Pr. Zineb Besri

Table des matières

Introduction	2
1 Architecture de la solution	3
1.1 Schéma d'architecture	3
1.2 Composants de l'architecture	3
2 Docker Compose	4
2.1 Fichier docker-compose.yml	4
2.2 Explication des instructions Docker Compose	5
3 Commandes utilisées	6
3.1 Lancement de l'architecture	6
3.2 Vérification des services	6
4 Captures d'exécution	7
4.1 Capture 1 : docker-compose up -d	7
4.2 Capture 2 : docker-compose ps	7
4.3 Capture 3 : docker ps (conteneurs actifs)	7
4.4 Capture 4 : Service Ticket - Swagger	8
4.5 Capture 5 : Service Utilisateur - Swagger	8
4.6 Checklist de déploiement	9
Conclusion	10

Introduction

L'objectif de ce livrable est de **gérer plusieurs microservices ensemble** à l'aide de Docker Compose. Docker Compose permet de définir et d'exécuter des applications multi-conteneurs avec un seul fichier de configuration.

Les composants de l'architecture sont :

1. Service Ticket (API REST de gestion des tickets)
2. Service Utilisateur (API REST de gestion des utilisateurs)
3. Réseau dédié pour la communication inter-services

Les tests sont effectués via l'interface Swagger pour valider le bon fonctionnement des deux microservices.

1 Architecture de la solution

1.1 Schéma d'architecture

Architecture Docker Compose		
Service	Port	Rôle
ticket-service	8080	Gestion des tickets (CRUD)
user-service	8081	Gestion des utilisateurs
↕ Réseau bridge : microservices-network ↕		

1.2 Composants de l'architecture

Service	Port	Rôle
ticket-service	8080	API de gestion des tickets
user-service	8081	API de gestion des utilisateurs
microservices-network	-	Réseau bridge pour la communication

2 Docker Compose

2.1 Fichier docker-compose.yml

```
version: '3.8'

services:

    #           Service Ticket

    ticket-service:
        build:
            context: ./ticket-microservice
            dockerfile: Dockerfile
        container_name: ticket-service
        ports:
            - "8080:8080"
        environment:
            - SERVER_PORT=8080
            - SPRING_APPLICATION_NAME=ticket-microservice
            - DB_USERNAME=sa
            - DB_PASSWORD=
        networks:
            - microservices-network
        restart: on-failure
        healthcheck:
            test: ["CMD-SHELL", "wget -q --spider http://localhost:8080/api/tickets || exit 1"]
            interval: 30s
            timeout: 10s
            retries: 3

    #           Service Utilisateur

    user-service:
        build:
            context: ./user-microservice
            dockerfile: Dockerfile
        container_name: user-service
        ports:
            - "8081:8081"
        environment:
            - SERVER_PORT=8081
            - SPRING_APPLICATION_NAME=user-microservice
            - DB_USERNAME=sa
            - DB_PASSWORD=
```

```
networks:
  - microservices-network
restart: on-failure
healthcheck:
  test: ["CMD-SHELL", "wget -q --spider http://localhost:8081/api
    /users || _exit 1"]
  interval: 30s
  timeout: 10s
  retries: 3

#       Réseau partag

networks:
  microservices-network:
    driver: bridge
```

Listing 2.1 – docker-compose.yml - Orchestration complète

2.2 Explication des instructions Docker Compose

Instruction	Description
version	Version de la syntaxe Docker Compose
services	Définition des conteneurs
build	Chemin vers le Dockerfile et contexte de build
container_name	Nom personnalisé du conteneur
ports	Mapping des ports (hôte :conteneur)
environment	Variables d'environnement
networks	Réseau auquel le conteneur se connecte
restart	Politique de redémarrage
healthcheck	Vérification de l'état du service

3 Commandes utilisées

3.1 Lancement de l'architecture

```
# Démarrer tous les services
docker-compose up -d

# Voir l'état des services
docker-compose ps

# Voir les logs en temps réel
docker-compose logs -f

# Arrêter tous les services
docker-compose down

# Reconstruire les images et démarrer
docker-compose up -d --build
```

Listing 3.1 – Commandes Docker Compose

3.2 Vérification des services

```
# Vérifier que les conteneurs tournent
docker ps

# Tester le service Ticket
curl http://localhost:8080/api/tickets

# Tester le service Utilisateur
curl http://localhost:8081/api/users
```

Listing 3.2 – Vérification des microservices

4 Captures d'exécution

4.1 Capture 1 : docker-compose up -d

```
PS C:\Users\user\Documents\sgitu-microservices> docker-compose up -d
time="2026-06-07T15:42:33+02:00" level=warning msg="C:\\Users\\user\\Documents\\sgitu-microservices\\docker-compose.yml: the attribute 'version' is obsolete, it
will be ignored, please remove it to avoid potential confusion"
[+] up 2/2
✓ Container ticket-service Running 0.0s
✓ Container user-service Running 0.0s
```

4.2 Capture 2 : docker-compose ps

```
PS C:\Users\user\Documents\ticket-microservice> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
c5d20ddd389e   ticket-microservice:latest         "java -jar app.jar"     30 seconds ago Up 29 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
ticket-service
cc947e75b8ba   grafana/grafana:latest            "/run.sh"               18 hours ago  Up 14 minutes 0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
g6-local-grafana
42716d90bde2   sgitu/payment-service:1.0.0       "java -XX:MaxRAMPerc..." 18 hours ago  Up 14 minutes (healthy) 0.0.0.0:8086->8086/tcp, [::]:8086->8086/tcp
payment-service
96bbfd23b77   phpmyadmin:latest                 "/docker-entrypoint.s..." 18 hours ago  Up 14 minutes 0.0.0.0:8896->80/tcp, [::]:8896->80/tcp
phpmyadmin-payment
118fe58c90a0   mysql:8.0                         "docker-entrypoint.s..." 18 hours ago  Up 14 minutes (healthy) 0.0.0.0:3316->3306/tcp, [::]:3316->3306/tcp
payment-db
b4495d9880c7   projet_ml_healthy_illusion-ui     "streamlit run app.p..." 7 days ago    Up 14 minutes 0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
healthy_illusion-ui
acde8de8e8ab   projet_ml_healthy_illusion-api    "uvicorn main:app --..." 7 days ago    Up 14 minutes (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
healthy_illusion-api
```

4.3 Capture 3 : docker ps (conteneurs actifs)

```
PS C:\Users\user\Documents\sgitu-microservices> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
NAMES
27f65c537435   sgitu-microservices-ticket-service "java -jar app.jar"     15 minutes ago Up 15 minutes (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/t
cp ticket-service
dfcc52042256   sgitu-microservices-user-service  "java -jar app.jar"     15 minutes ago Up 15 minutes (healthy) 0.0.0.0:8081->8081/tcp, [::]:8081->8081/t
cp user-service
cc947e75b8ba   grafana/grafana:latest            "/run.sh"               22 hours ago  Up 4 hours 0.0.0.0:3000->3000/tcp, [::]:3000->3000/t
cp g6-local-grafana
42716d90bde2   sgitu/payment-service:1.0.0       "java -XX:MaxRAMPerc..." 22 hours ago  Up 4 hours (healthy) 0.0.0.0:8086->8086/tcp, [::]:8086->8086/t
cp payment-service
96bbfd23b77   phpmyadmin:latest                 "/docker-entrypoint.s..." 22 hours ago  Up 4 hours 0.0.0.0:8896->80/tcp, [::]:8896->80/tcp
phpmyadmin-payment
118fe58c90a0   mysql:8.0                         "docker-entrypoint.s..." 22 hours ago  Up 4 hours (healthy) 0.0.0.0:3316->3306/tcp, [::]:3316->3306/t
cp payment-db
b4495d9880c7   projet_ml_healthy_illusion-ui     "streamlit run app.p..." 7 days ago    Up 4 hours 0.0.0.0:8501->8501/tcp, [::]:8501->8501/t
cp healthy_illusion-ui
```


4.4 Capture 4 : Service Ticket - Swagger

Gestion des Tickets API pour la gestion des tickets

GET /api/tickets Lister tous les tickets

Parameters

No parameters

Execute Clear

Responses

Code Details

200

Response body

Response headers

connection: keep-alive
content-type: application/json
date: Sun, 07 Jun 2020 13:44:49 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers

Responses

Code Description Links

200 OK No links

Media type

*/

Controls Accept header

Example Value Schema

```
{
  "id": 0,
  "title": "string",
  "description": "string",
  "status": "string",
  "priority": "string",
  "createdAt": "2020-06-07T13:44:49.137Z",
  "clientName": "string",
  "clientEmail": "string"
}
```

4.5 Capture 5 : Service Utilisateur - Swagger

Gestion des Utilisateurs API pour la gestion des utilisateurs

GET /api/users Lister tous les utilisateurs

Parameters

No parameters

Execute Clear

Responses

Code Details

200

Response body

Response headers

connection: keep-alive
content-type: application/json
date: Sun, 07 Jun 2020 13:33:27 GMT
keep-alive: timeout=60
transfer-encoding: chunked
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers

Responses

Code Description Links

200 OK No links

Media type

*/

Controls Accept header

Example Value Schema

```
{
  "id": 0,
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "role": "string",
  "createdAt": "2020-06-07T13:33:30.775Z"
}
```

4.6 Checklist de déploiement

- ☐ `docker-compose.yml` correctement configuré
- ☐ Les 2 services sont construits sans erreur
- ☐ Service Ticket (8080) répond correctement
- ☐ Service Utilisateur (8081) répond correctement
- ☐ Les services communiquent via le réseau dédié
- ☐ Les healthchecks sont fonctionnels

Conclusion

Résumé des travaux réalisés :

1. **Configuration Docker Compose** : Fichier `docker-compose.yml` avec 2 services
2. **Réseau dédié** : Isolation et communication inter-services
3. **Healthchecks** : Vérification de l'état des services
4. **Orchestration** : Commandes `up`, `ps`, `logs`, `down`

Bonnes pratiques Docker Compose retenues :

- Utilisation d'un réseau bridge dédié
- Healthchecks pour surveiller l'état des services
- Politique de redémarrage `on-failure`
- Variables d'environnement pour la configuration